

How to Use the Flutter Reusable Core Kit

A practical installation and usage guide for starting new Flutter projects with shared theme, language, widgets, validators, services, and clean architecture conventions.

Created for:	Reusable files extracted from Fruite Hub and PharmaChain patterns
Use with:	Flutter, Cubit/Bloc, GetIt, SharedPreferences, EasyLocalization, Firebase/Supabase when needed
Generated:	2026-05-21

1. What this kit gives you

The ZIP is not a published Dart package. It is a reusable starter core that you copy into a new Flutter project. It gives you a clean base so you do not rebuild the same services, widgets, theme, and localization setup every time.

- Shared core folder for theme, language, dependency injection, cache, services, extensions, validators, errors, and common widgets.
- Settings feature for light/dark mode and English/Arabic language switching.
- Reusable UI components such as buttons, text fields, safe network image, empty/error/loading states, cards, status chips, bottom sheet header, and page padding.
- Service patterns for SharedPreferences, Firebase Auth, Firestore, and Supabase Storage when your app needs them.
- Clean architecture guidance compatible with your preferred feature structure: data and presentation only.

Important

Copy only what your project needs. For example, if a project does not use Supabase, do not keep the Supabase dependency or service registration.

2. Copy the files into a new project

After extracting the ZIP, copy these folders into your Flutter project root:

```
flutter_reusable_core_kit_from_fruite_pharmacy/  
  lib/core/          -> your_project/lib/core/  
  lib/features/settings/ -> your_project/lib/features/settings/  
  assets/translations/ -> your_project/assets/translations/
```

Your new project should then look like this:

```
your_project/  
  lib/  
    core/  
      cache/  
      di/  
      errors/  
      extensions/  
      language/  
      models/  
      routing/  
      services/  
      theme/  
      utils/  
      widgets/  
    features/  
      settings/  
        data/  
        presentation/  
  assets/  
    translations/  
      en.json  
      ar.json
```

3. Add the required dependencies

Open pubspec_dependencies.yaml from the kit and copy the dependencies you need into your project pubspec.yaml. A typical setup is:

```
dependencies:
  flutter:
    sdk: flutter
  flutter_bloc: ^9.1.0
  get_it: ^8.0.3
  shared_preferences: ^2.5.3
  easy_localization: ^3.0.7

# Keep these only if the app needs Firebase:
  firebase_core: ^3.13.0
  firebase_auth: ^5.5.2
  cloud_firestore: ^5.6.6

# Keep this only if the app needs Supabase Storage:
  supabase_flutter: ^2.8.4
```

Then run:

```
flutter pub get
```

4. Register translation assets

In your project pubspec.yaml, register the translations folder:

```
flutter:
  assets:
    - assets/translations/
```

The ZIP does not include font files. If your theme references a font family, add your own fonts or change the font name in the theme files.

5. Initialize the kit in main.dart

Use EasyLocalization and setupServiceLocator before running the app:

```
import 'package:easy_localization/easy_localization.dart';
import 'package:flutter/material.dart';

import 'core/di/service_locator.dart';
import 'my_app.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await EasyLocalization.ensureInitialized();

  await setupServiceLocator();

  runApp(
    EasyLocalization(
      supportedLocales: const [Locale('en'), Locale('ar')],
      path: 'assets/translations',
      fallbackLocale: const Locale('en'),
      child: const MyApp(),
    ),
  );
}
```

6. Connect theme and language in MyApp

If you use normal MaterialApp, connect themeMode and localization like this:

```

import 'package:easy_localization/easy_localization.dart';
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import 'core/di/service_locator.dart';
import 'core/theme/app_theme.dart';
import 'features/settings/presentation/cubit/app_preferences_cubit.dart';
import 'home_screen.dart';

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return BlocProvider(
      create: (_) => getIt<AppPreferencesCubit>().loadPreferences(),
      child: BlocBuilder<AppPreferencesCubit, AppPreferencesState>(
        builder: (context, state) {
          return MaterialApp(
            debugShowCheckedModeBanner: false,
            locale: context.locale,
            supportedLocales: context.supportedLocales,
            localizationsDelegates: context.localizationDelegates,
            theme: AppTheme.light,
            darkTheme: AppTheme.dark,
            themeMode: state.themeMode,
            home: const HomeScreen(),
          );
        },
      ),
    );
  }
}

```

If your app uses `go_router`, use `MaterialApp.router` with the same theme and localization properties.

7. Use the settings Cubit

The `AppPreferencesCubit` should own language and theme actions. UI widgets should only call Cubit methods.

```

// Toggle theme
context.read<AppPreferencesCubit>().toggleTheme();

// Change language
context.read<AppPreferencesCubit>().changeLanguage(
  context: context,
  locale: const Locale('ar'),
);

```

8. Use common widgets

Button:

```

AppButton(
  text: 'save'.tr(),
  onPressed: () {},
)

```

Text field:

```

AppTextField(
  controller: nameController,
  labelText: 'name'.tr(),
  validator: AppValidators.required,
)

```

Password field:

```
AppPasswordField(  
  controller: passwordController,  
  labelText: 'password'.tr(),  
  validator: AppValidators.password,  
)
```

Safe network image:

```
AppNetworkImage(  
  imageUrl: product.imageUrl,  
  width: 80,  
  height: 80,  
)
```

Bottom sheet header:

```
AppBottomSheetHeader(  
  title: 'add_item'.tr(),  
  onPressed: () => Navigator.pop(context),  
)
```

9. Recommended feature structure

For every feature in new apps, keep the structure clean and small:

```
lib/features/<feature>/  
  data/  
    data_source/  
    models/  
    repos/  
  presentation/  
    cubit/  
    refactor/  
    screens/  
    widgets/
```

- UI files should display state and call Cubit actions only.
- Cubit should own screen behavior, filtering, selected values, loading flags, and error keys.
- Data sources should own backend calls, validation, transactions, and normalization.
- Repos should be thin and delegate to data sources.
- Keep UI files near 100-120 lines when possible; split sections into refactor/widgets files.

10. Files to customize first

After copying the kit, these are the first files you should edit in each new project:

File	What to change
lib/core/theme/app_colors.dart	Main brand colors and semantic colors.
lib/core/theme/app_theme.dart	Material theme, light/dark theme, inputs, cards, buttons.
lib/core/language/lang_keys.dart	Add project-specific localization keys.
assets/translations/en.json	English strings.
assets/translations/ar.json	Arabic strings.
lib/core/routing/app_routes.dart	Project routes and route helper methods.

lib/core/di/service_locator.dart

Register Cubits, repos, data sources, services.

11. Dependency injection pattern

Keep service locator setup grouped and simple. Register services first, then data sources, repos, and Cubits.

```
final getIt = GetIt.instance;

Future<void> setupServiceLocator() async {
  final prefs = await SharedPreferences.getInstance();

  getIt.registerLazySingleton<AppCacheService>(
    () => AppCacheService(prefs),
  );

  getIt.registerLazySingleton<SettingsLocalDataSource>(
    () => SettingsLocalDataSource(getIt<AppCacheService>()),
  );

  getIt.registerLazySingleton<SettingsRepo>(
    () => SettingsRepo(getIt<SettingsLocalDataSource>()),
  );

  getIt.registerFactory<AppPreferencesCubit>(
    () => AppPreferencesCubit(getIt<SettingsRepo>()),
  );
}
```

12. Checklist for every new project

- Copy lib/core and lib/features/settings into the project.
- Copy assets/translations and register it in pubspec.yaml.
- Add only the dependencies you need.
- Run flutter pub get.
- Initialize EasyLocalization and setupServiceLocator in main.dart.
- Wrap MyApp with AppPreferencesCubit and connect themeMode.
- Replace hardcoded strings with localization keys.
- Use AppNetworkImage for remote images to avoid broken-image overflow.
- Use AppPagePadding and AppScaffold for consistent responsive spacing.
- Keep feature UI dumb; move behavior to Cubit and data rules to data sources.
- Run flutter analyze locally after integration.

Final advice

Treat this kit as your project foundation, not a finished app. Copy it, rename/customize theme and routes, remove unused services, then build each feature using the same data/presentation structure.

13. Common problems and fixes

Problem	Fix
Missing asset error for translations	Add assets/translations/ to pubspec.yaml and run flutter pub get.

Missing Firebase initialization	Call <code>Firebase.initializeApp</code> before using Firebase services, or remove Firebase services if unused.
Font not found	Add your own fonts to <code>pubspec.yaml</code> or change the font family in <code>AppTextStyles</code> .
Cubit not found in <code>GetIt</code>	Register the Cubit in <code>service_locator.dart</code> before using <code>getIt()</code> .
Language does not change	Make sure <code>EasyLocalization</code> wraps <code>MyApp</code> and <code>changeLanguage</code> calls <code>context.setLocale</code> .
Dark mode does not apply	Make sure <code>MaterialApp</code> uses <code>theme</code> , <code>darkTheme</code> , and <code>themeMode</code> from <code>AppPreferencesCubit</code> .